LESSON 25

挨个 试遍

for x in range(...)

枚举：把可能都试一遍

学完你能让程序把所有可能一个个试过去，找出鸡兔同笼这类题的答案。

行李箱的密码忘了怎么办?

三位数的密码锁

000 到 999

一个个拨过去

总能打开

不知道答案，就把所有可能都试一遍。最多拨 1000 次，一定能找到。

人拨很累，计算机试起来飞快。这种办法叫枚举，也叫“穷举”：挨个列出来，一个个检查。

哪个数自己乘自己是 144?

square.py

```
1 for x in range(1, 145):
2     if x * x == 144:
3         print(x)
```

每个 x 都检查一下：x * x 是 144 吗？是就输出。试到 12 时对上了。

要试	144 次
屏幕上	12

你来当计算机，试一试

try.py

```
1 x = 10
2 print(x * x)
3 print(x * x == 144)
```

点黄色的数字，改完结果马上变。

第 2 行：x 乘 x

100

第 3 行：对上了吗

False

从 1 开始一个个改，改到几才是 True？你一个个试，就是在做枚举。程序替你试，快得多。

猜一猜：把范围改成 range(1, 12)，输出什么？

square.py

```
1 for x in range(1, 12):
2     if x * x == 144:
3         print(x)
```

什么也
不输出

range(1, 12) 只试到 11，不含 12。答案不在范围里，就一个也找不到，而且不报错。
枚举最要紧的是范围：答案可能出现的地方，一个都不能漏。

小测：1 到 100 都要试，range 怎么写？

1 `range(1, 100)`

2 `range(1, 101)`

3 `range(0, 100)`

4 `range(100, 1)`

答对了！从 1 开始，终点写 101，正好试到 100。

枚举分三步

1

定范围

答案可能是哪些？一个都不能漏

2

一个个试

用 `for` 循环把它们挨个拿出来

3

判断对不对

用 `if` 检查，对上了就输出

想不出巧办法时，先想想能不能挨个试。题目给的条件，就是第 3 步的判断。

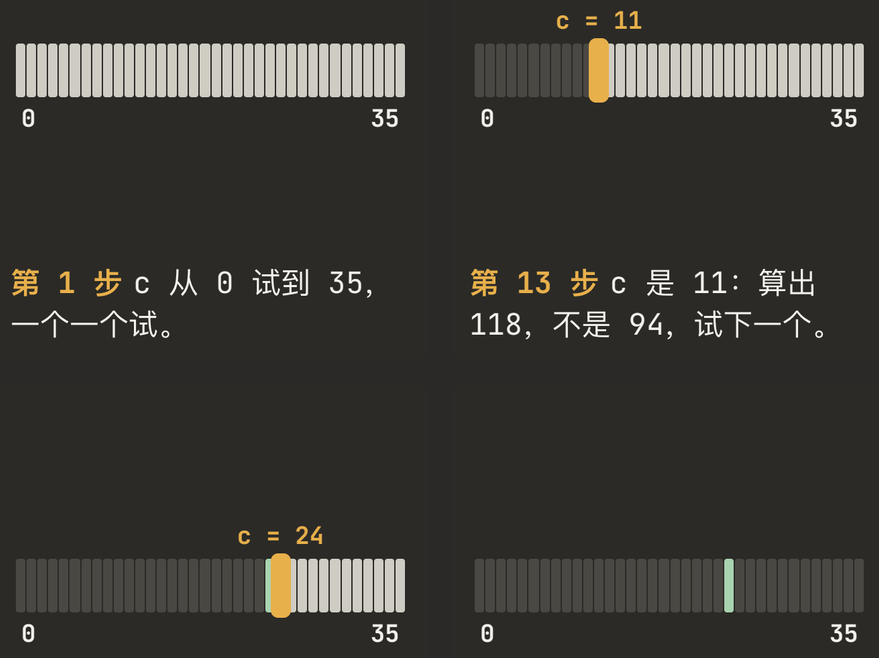
例题：鸡兔同笼

cage.py

```
heads = int(input())
feet = int(input())
for c in range(heads + 1):
    r = heads - c
    if c * 2 + r * 4 == feet:
        print("鸡", c, "兔", r)
```

35 个头、94 只脚，c 是鸡，r 是兔。

鸡一只 2 只脚，兔一只 4 只脚。



第 1 步 c 从 0 试到 35，一个一个试。

第 13 步 c 是 11：算出 118，不是 94，试下一个。

第 26 步 c 是 24：算出 92，不是 94，试下一个。

第 38 步 试完了，一共试了 36 个，对上的只有 23。

小测：35 个头时，例题的循环一共试了几次?

1 35 次

2 36 次

3 94 次

4 24 次

答对了！range(heads + 1) 是 0 到 35，一共 36 个。

动手改一改

一个也没找到时，说一声。

没有答案时说一声

脚数是单数，或者太多太少，就怎么也凑不上。在例题上改：一种答案也没有时，输出“没有答案”。输入还是两行：头数和脚数。

输入 35 和 94

鸡 23 兔 12

输入 10 和 21

没有答案

第 22 课的 done 记着“输对了没有”，这里也可以记着“找到过没有”。

这节课常见的三种错误

划线的写法只阅读，不要照抄

~~`range(heads)`~~	**不报错**	漏了“全是鸡”这一种
~~`range(1, heads + 1)`~~	**不报错**	漏了“一只鸡也没有”
循环里写“没有答案”	**不报错**	每试错一次就说一遍

枚举的错大多不报错，只是少了答案或多了废话。用两头的数据试一试：全是鸡、全是兔。

找错：10 个头 20 只脚，全是鸡，却没输出

bug.py（只阅读）

```
heads = int(input())
feet = int(input())
for c in range(heads):
    r = heads - c
    if c * 2 + r * 4 == feet:
        print("鸡", c, "兔", r)
```

点出该改的那一行。

点代码里的一行

自己挑战

百钱买百鸡

一道一千多年前的古题：公鸡 5 元一只，母鸡 3 元一只，小鸡 1 元 3 只。读入 n（1 到 300 的整数），用 n 元正好买 n 只鸡，小鸡要整 3 只地买。
每种买法输出一行，公鸡从少到多，公鸡一样时母鸡从少到多。一种也没有就输出“没有答案”。

输入

```
100
```

输出

```
公鸡 0 母鸡 25 小鸡 75
公鸡 4 母鸡 18 小鸡 78
公鸡 8 母鸡 11 小鸡 81
公鸡 12 母鸡 4 小鸡 84
```

先自己写，卡住了再翻下一页。

卡住了？一次看一条提示

提示 1　定范围：公鸡最多 `n // 5` 只，母鸡最多 `n // 3` 只。用第 18 课的两层循环，外层试公鸡，内层试母鸡。

提示 2　只数凑满 `n`，小鸡就是 `n - 公鸡 - 母鸡`。小鸡不能是负数，还要是 3 的倍数：`% 3 == 0`。

提示 3　再检查钱：`公鸡 × 5 + 母鸡 × 3 + 小鸡 // 3` 等于 `n` 就输出。没找到过就说“没有答案”，和动手题一样。

参考答案：百钱买百鸡

hundred.py

```python
n = int(input())
found = False
for cock in range(n // 5 + 1):
    for hen in range(n // 3 + 1):
        chick = n - cock - hen
        if chick >= 0 and chick % 3 == 0:
            if cock * 5 + hen * 3 + chick // 3 == n:
                print("公鸡", cock, "母鸡", hen, "小鸡", chick)
                found = True
if not found:
    print("没有答案")
```

两层循环试公鸡和母鸡，小鸡是剩下的，钱对上就输出。

造一个“挨个试”的小程序

自己出一道题，让程序挨个试出答案。

先做到

找出 1 到 100 里你的幸运数，比如个位是 7、又是 3 的倍数。

选你喜欢的

三位数里百位和个位一样、各位加起来是 10 的；两个数和是 20、积是 96。

还想更厉害

数一数一共试了几次，再想办法缩小范围，少试几次。

没有标准答案。先说清它要做什么，再试两组输入，一组试边界。

和同桌换着玩

1

先猜后跑

先听同桌的题，自己猜答案，再运行看看

2

说一个喜欢的

告诉同桌，哪道题出得最有意思

3

提一个想法

范围有没有漏？能不能少试几次？

听完再改一版，让题目更好玩。

说得清吗?

Q1	**枚举分哪三步?**	定范围，一个个试，判断对不对。
Q2	**范围漏了会怎样?**	不报错，只是找不到漏掉的答案。
Q3	**怎样在没答案时说一声?**	用 `found` 记着找到过没有，循环结束后再判断。

速查卡

`for x in range(1, 101):`	1 到 100 挨个试
`if 条件:`	对上了就输出
`range(heads + 1)`	从 0 到 heads，两头都要试
`found = False`	记着找到过没有
两个未知数	两层循环，或者用一个算出另一个

这些后面再学

缩小范围

先想清楚哪些不可能，就不用试

二分查找

排好序的数，每次去掉一半，试得更少

回溯

走不通就退一步，换条路再试

今天学会了挨个试。下节课学二分查找：同样是找，每次能排除一半。

谢谢！

喜欢这节课？欢迎来 GitHub 和我交流。

 python-for-youth

下一课：二分查找